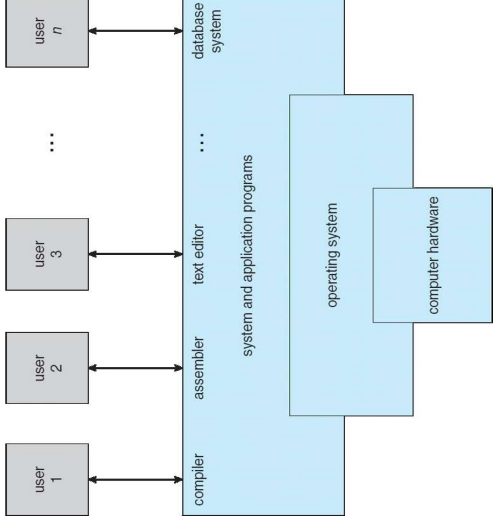


Computer System Structure

Computer system can be divided into four components:

- **Hardware** : provides basic computing resources
CPU, memory, I/O devices
- **Operating system:**
Controls and coordinates use of hardware among various applications and users
- **Application programs** : define the ways in which the system resources are used to solve the computing problems of the users
Word processors, compilers, web browsers, database systems, video games
- **Users:**
People, machines, other computers



Definition of Operating System

- OS is a resource allocator
- Manages all resources: Decides between conflicting requests for efficient and fair resource use
- OS is a control program: Controls execution of programs to prevent errors and improper use of the computer

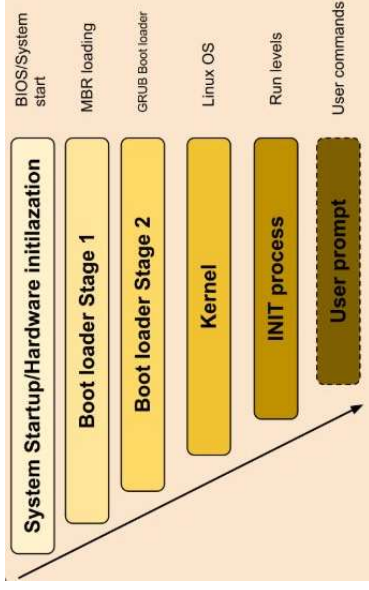
No universally accepted definition
“Everything a vendor ships when you order an operating system” is a good approximation
But varies wildly

“The one program running at all times on the computer” is the kernel.

Everything else is either
a system program (ships with the operating system) , or
an application program.

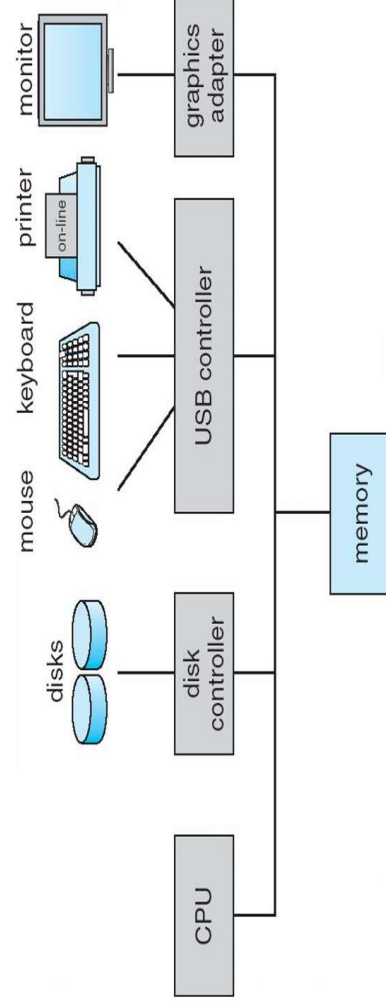
Computer Startup (Booting Process)

- bootstrap program is loaded at power-up or reboot
- Typically stored in ROM or EPROM, generally known as firmware
- Initializes all aspects of system
- Loads operating system kernel and starts execution



Computer System Organization

- Computer-system operation
 - One or more CPUs, device controllers connect through common bus providing access to shared memory
 - Concurrent execution of CPUs and devices competing for memory cycles



Computer-System Operation

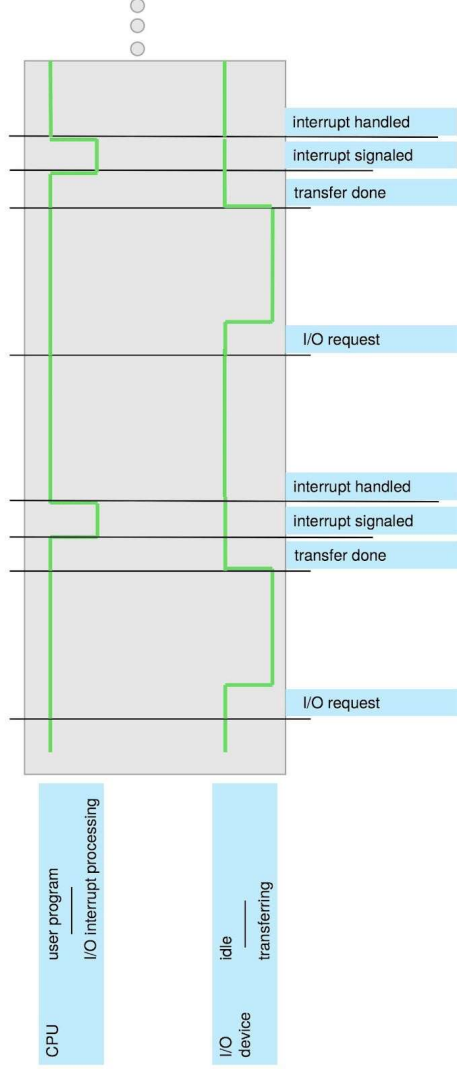
- I/O devices and the CPU can execute concurrently
- Each device controller is in charge of a particular device type
- Each device controller has a local buffer
- CPU moves data from/to main memory to/from local buffers
- I/O is from the device to local buffer of controller
- Device controller informs CPU that it has finished its operation by causing an interrupt

Common Functions of Interrupts

- Interrupt transfers control to the interrupt service routine generally, through the **interrupt vector**, which contains the addresses of all the service routines
- Interrupt architecture must save the address of the interrupted instruction
- A **trap** or **exception** is a software-generated interrupt caused either by an error or a user request
- An operating system is **interrupt driven**

Interrupt Handling

- The operating system preserves the state of the CPU by storing registers and the program counter
- Determines which type of interrupt has occurred:
 - **polling**
 - **vectored** interrupt system
- Separate segments of code determine what action should be taken for each type of interrupt



I/O Structure

- After I/O starts, control returns to user program only upon I/O completion
 - Wait instruction idles the CPU until the next interrupt
 - Wait loop (contention for memory access)
 - At most one I/O request is outstanding at a time, no simultaneous I/O processing
- After I/O starts, control returns to user program without waiting for I/O completion
 - **System call** – request to the OS to allow user to wait for I/O completion
 - **Device-status table** contains entry for each I/O device indicating its type, address, and state
 - OS indexes into I/O device table to determine device status and to modify table entry to include interrupt

Storage Definitions and Notation Review

The basic unit of computer storage is the **bit**. A bit can contain one of two values, 0 and 1. All other storage in a computer is based on collections of bits. Given enough bits, it is amazing how many things a computer can represent: numbers, letters, images, movies, sounds, documents, and programs, to name a few. A **byte** is 8 bits, and on most computers it is the smallest convenient chunk of storage. For example, most computers don't have an instruction to move a bit but do have one to move a byte. A less common term is **word**, which is a given computer architecture's native unit of data. A word is made up of one or more bytes. For example, a computer that has 64-bit registers and 64-bit memory addressing typically has 64-bit (8-byte) words. A computer executes many operations in its native word size rather than a byte at a time.

Computer storage, along with most computer throughput, is generally measured and manipulated in bytes and collections of bytes.

A **kilobyte**, or **KB**, is $1,024$ bytes
a **megabyte**, or **MB**, is $1,024^2$ bytes
a **gigabyte**, or **GB**, is $1,024^3$ bytes
a **terabyte**, or **TB**, is $1,024^4$ bytes
a **petabyte**, or **PB**, is $1,024^5$ bytes

Computer manufacturers often round off these numbers and say that a megabyte is 1 million bytes and a gigabyte is 1 billion bytes. Networking measurements are an exception to this general rule; they are given in bits (because networks move data a bit at a time).

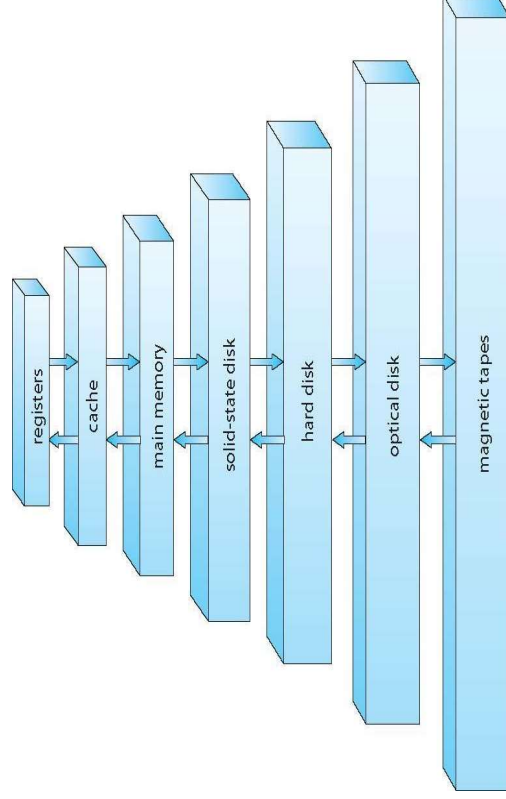
Storage Structure

- Main memory – only large storage media that the CPU can access directly
 - **Random access**
 - Typically **volatile**
- Secondary storage – extension of main memory that provides large **nonvolatile** storage capacity
- Hard disks – rigid metal or glass platters covered with magnetic recording material
 - Disk surface is logically divided into **tracks**, which are subdivided into **sectors**
 - The **disk controller** determines the logical interaction between the device and the computer
- **Solid-state disks** – faster than hard disks, nonvolatile
 - Various technologies
 - Becoming more popular

Storage Hierarchy

- Storage systems organized in hierarchy
 - Speed
 - Cost
 - Volatility
- Caching** – copying information into faster storage system; main memory can be viewed as a cache for secondary storage
- Device Driver** for each device controller to manage I/O
 - Provides uniform interface between controller and kernel

Storage-Device Hierarchy



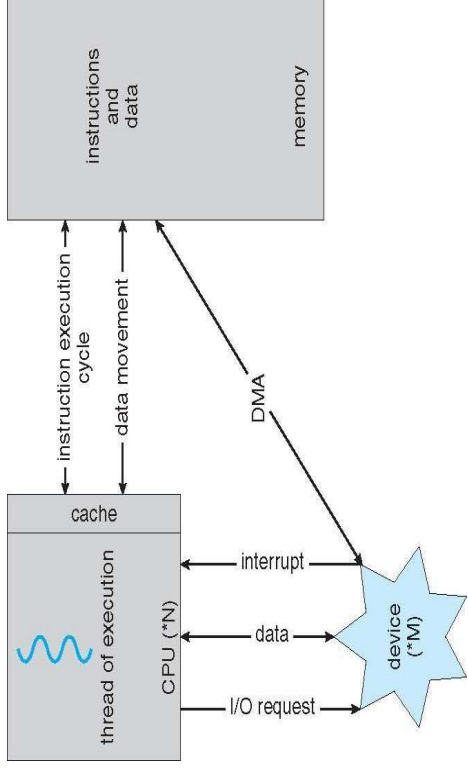
Caching

- Important principle, performed at many levels in a computer (in hardware, operating system, software)
- Information in use copied from slower to faster storage temporarily
- Faster storage (cache) checked first to determine if information is there
 - If it is, information used directly from the cache (fast)
 - If not, data copied to cache and used there
- Cache smaller than storage being cached
- Cache management important design problem
- Cache size and replacement policy

Direct Memory Access Structure

- Used for high-speed I/O devices able to transmit information at close to memory speeds
- Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention
- Only one interrupt is generated per block, rather than the one interrupt per byte

How a Modern Computer Works

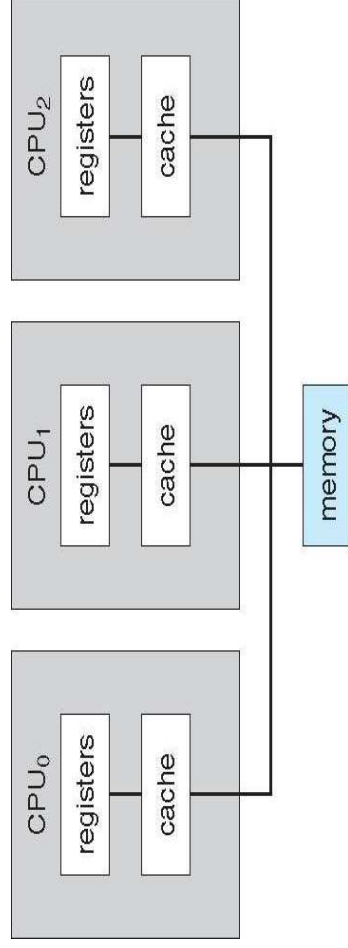


A von Neumann architecture

Computer-System Architecture

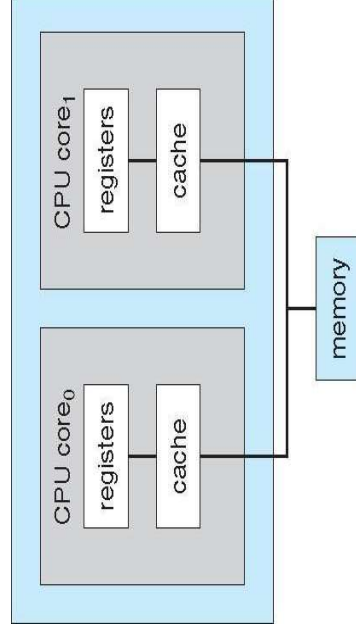
- Most systems use a single general-purpose processor
- Most systems have special-purpose processors as well
- **Multiprocessors** systems growing in use and importance
 - Also known as **parallel systems**, **tightly-coupled systems**
 - Advantages include:
 1. **Increased throughput**
 2. **Economy of scale**
 3. **Increased reliability** – graceful degradation or fault tolerance
 - Two types:
 1. **Asymmetric Multiprocessing** – each processor is assigned a specific task.
 2. **Symmetric Multiprocessing** – each processor performs all tasks

Symmetric Multiprocessing Architecture



A Dual-Core Design

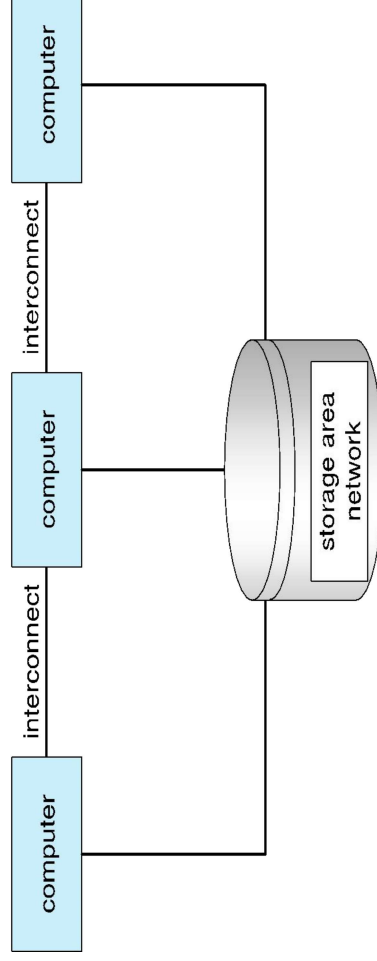
- Multi-chip and **multicore**
- Systems containing all chips
 - Chassis containing multiple separate systems



Clustered Systems

- Like multiprocessor systems, but multiple systems working together
 - Usually sharing storage via a **Storage Area Network (SAN)**
 - Provides a **high-availability** service which survives failures
 - **Asymmetric clustering** has one machine in hot-standby mode
 - **Symmetric clustering** has multiple nodes running applications, monitoring each other
 - Some clusters are for **High Performance Computing (HPC)**
 - Applications must be written to use **parallelization**
 - Some have **Distributed Lock Manager (DLM)** to avoid conflicting operations

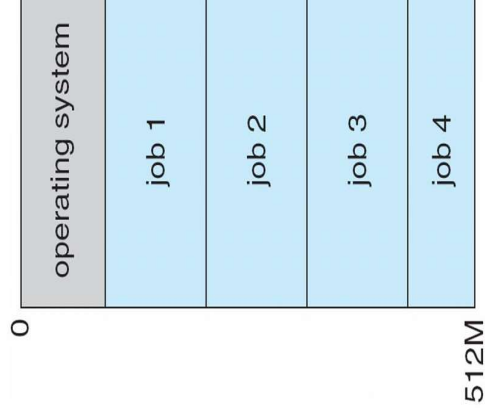
Clustered Systems



Operating System Structure

- **Multiprogramming (Batch system)** needed for efficiency
 - Single user cannot keep CPU and I/O devices busy at all times
 - Multiprogramming organizes jobs (code and data) so CPU always has one to execute
 - A subset of total jobs in system is kept in memory
 - One job selected and run via **job scheduling**
 - When it has to wait (for I/O for example), OS switches to another job
- **Timesharing (multitasking)** is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing
 - **Response time** should be < 1 second
 - Each user has at least one program executing in memory ☐ **process**
 - If several jobs ready to run at the same time ☐ **CPU scheduling**
 - If processes don't fit in memory, **swapping** moves them in and out to run
 - **Virtual memory** allows execution of processes not completely in memory

Memory Layout for Multiprogrammed System



Operating-System Operations

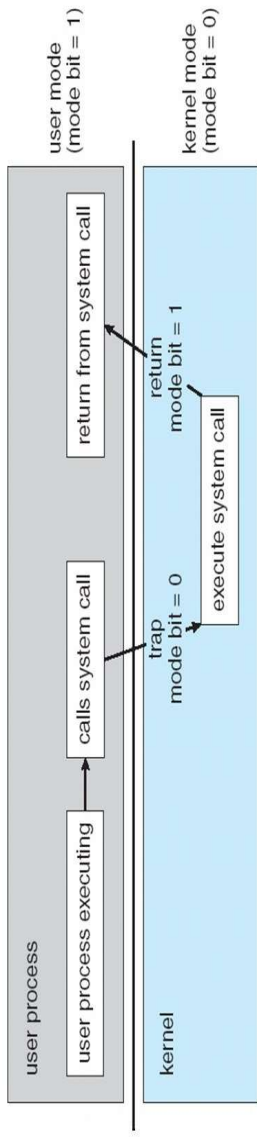
- **Interrupt driven** (hardware and software)
 - Hardware interrupt by one of the devices
 - Software interrupt (**exception** or **trap**):
 - Software error (e.g., division by zero)
 - Request for operating system service
 - Other process problems include infinite loop, processes modifying each other or the operating system

Operating-System Operations (cont.)

- **Dual-mode** operation allows OS to protect itself and other system components
 - **User mode** and **kernel mode**
 - **Mode bit** provided by hardware
 - Provides ability to distinguish when system is running user code or kernel code
 - Some instructions designated as **privileged**, only executable in kernel mode
 - System call changes mode to kernel, return from call resets it to user
- Increasingly CPUs support multi-mode operations
 - i.e. **virtual machine manager (VMM)** mode for guest **VMs**

Transition from User to Kernel Mode

- Timer to prevent infinite loop / process resources
 - Timer is set to interrupt the computer after some time period
 - Keep a counter that is decremented by the physical clock.
 - Operating system set the counter (privileged instruction)
 - When counter zero generate an interrupt
 - Set up before scheduling process to regain control or terminate program that exceeds allotted time



Process Management

- A process is a program in execution. It is a unit of work within the system. Program is a **passive entity**, process is an **active entity**.
- Process needs resources to accomplish its task
 - CPU, memory, I/O, files
 - Initialization data
- Process termination requires reclaim of any reusable resources
- Single-threaded process has one **program counter** specifying location of next instruction to execute
 - Process executes instructions sequentially, one at a time, until completion
- Multi-threaded process has one program counter per thread
- Typically system has many processes, some user, some operating system running concurrently on one or more CPUs
 - Concurrency by multiplexing the CPUs among the processes / threads

Process Management Activities

The operating system is responsible for the following activities in connection with process management:

- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication
- Providing mechanisms for deadlock handling

Memory Management

- To execute a program all (or part) of the instructions must be in memory
- All (or part) of the data that is needed by the program must be in memory.
- Memory management determines what is in memory and when
 - Optimizing CPU utilization and computer response to users
- Memory management activities
 - Keeping track of which parts of memory are currently being used and by whom
 - Deciding which processes (or parts thereof) and data to move into and out of memory
 - Allocating and deallocating memory space as needed

Storage Management

- OS provides uniform, logical view of information storage
 - Abstracts physical properties to logical storage unit - **file**
 - Each medium is controlled by device (i.e., disk drive, tape drive)
 - Varying properties include access speed, capacity, data-transfer rate, access method (sequential or random)
- File-System management
 - Files usually organized into directories
 - Access control on most systems to determine who can access what
 - OS activities include
 - Creating and deleting files and directories
 - Primitives to manipulate files and directories
 - Mapping files onto secondary storage
 - Backup files onto stable (non-volatile) storage media

Mass-Storage Management

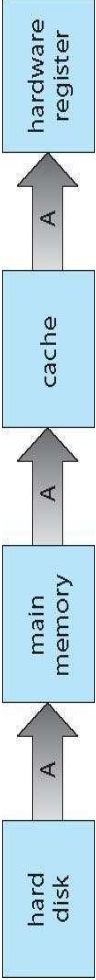
- Usually disks used to store data that does not fit in main memory or data that must be kept for a “long” period of time
- Proper management is of central importance
- Entire speed of computer operation hinges on disk subsystem and its algorithms
- OS activities
 - Free-space management
 - Storage allocation
 - Disk scheduling
- Some storage need not be fast
 - Tertiary storage includes optical storage, magnetic tape
 - Still must be managed – by OS or applications
 - Varies between WORM (write-once, read-many-times) and RW (read-write)

Performance of Various Levels of Storage

Level	1	2	3	4	5
Name	registers	cache	main memory	solid state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25 - 0.5	0.5 - 25	80 - 250	25,000 - 50,000	5,000,000
Bandwidth (MB/sec)	20,000 - 100,000	5,000 - 10,000	1,000 - 5,000	500	20 - 150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

Movement between levels of storage hierarchy can be explicit or implicit

Migration of data "A" from Disk to Register

- Multitasking environments must be careful to use most recent value, no matter where it is stored in the storage hierarchy
- 
- ```

graph LR
 A[A] --> B[A]
 B --> C[A]
 C --> D[A]
 D --> E[A]
 style A fill:#add8e6,stroke:#333,stroke-width:1px
 style B fill:#add8e6,stroke:#333,stroke-width:1px
 style C fill:#add8e6,stroke:#333,stroke-width:1px
 style D fill:#add8e6,stroke:#333,stroke-width:1px
 style E fill:#add8e6,stroke:#333,stroke-width:1px

```
- Multiprocessor environment must provide **cache coherency** in hardware such that all CPUs have the most recent value in their cache
  - Distributed environment situation even more complex
    - Several copies of a datum can exist

## I/O Subsystem

- One purpose of OS is to hide peculiarities of hardware devices from the user
- I/O subsystem responsible for
  - Memory management of I/O including
    - buffering (storing data temporarily while it is being transferred),
    - caching (storing parts of data in faster storage for performance),
    - spooling (the overlapping of output of one job with input of other jobs)
  - General device-driver interface
  - Drivers for specific hardware devices

## Protection and Security

- **Protection** – any mechanism for controlling access of processes or users to resources defined by the OS
- **Security** – defense of the system against internal and external attacks
  - Huge range, including denial-of-service, worms, viruses, identity theft, theft of service
  - Systems generally first distinguish among users, to determine who can do what
  - User identities (**user IDs**, security IDs) include name and associated number, one per user
  - User ID then associated with all files, processes of that user to determine access control
  - Group identifier (**group ID**) allows set of users to be defined and controls managed, then also associated with each process, file
  - **Privilege escalation** allows user to change to effective ID with more rights

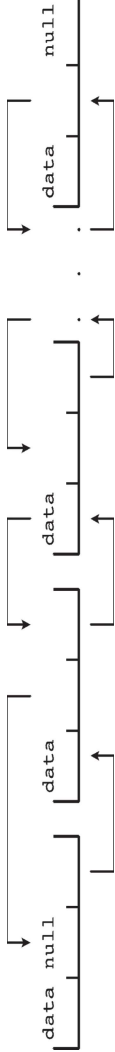
## Kernel Data Structures

Many similar to standard programming data structures

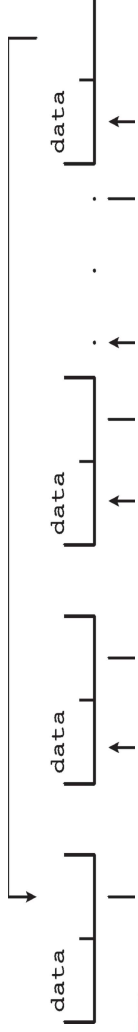
### *Singly linked list*



### *Doubly linked list*

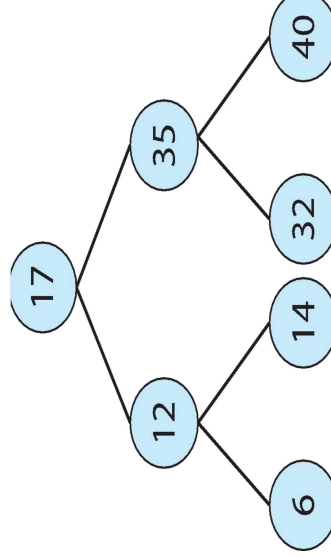


### *Circular linked list*



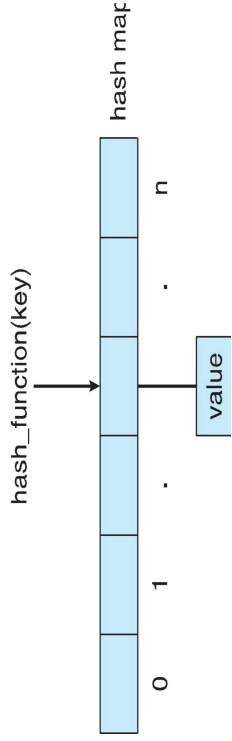
## Kernel Data Structures

- **Binary search tree**  
left <= right
  - Search performance is  $O(n)$
  - **Balanced binary search tree** is  $O(\lg n)$



## Kernel Data Structures

- Hash function can create a **hash map**



- **Bitmap** – string of  $n$  binary digits representing the status of  $n$  items
- Linux data structures defined in  
**include** files `<linux/list.h>`, `<linux/kfifo.h>`, `<linux/rbtree.h>`

## Computing Environments - Traditional

- Stand-alone general purpose machines
- But blurred as most systems interconnect with others (i.e., the Internet)
- **Portals** provide web access to internal systems
- **Network computers (thin clients)** are like Web terminals
- Mobile computers interconnect via **wireless networks**
- Networking becoming ubiquitous – even home systems use **firewalls** to protect home computers from Internet attacks

## Computing Environments - Mobile

- Handheld smartphones, tablets, etc
- What is the functional difference between them and a “traditional” laptop?
- Extra feature – more OS features (GPS, gyroscope)
- Allows new types of apps like *augmented reality*
- Use IEEE 802.11 wireless, or cellular data networks for connectivity
- Leaders are **Apple iOS** and **Google Android**

## Computing Environments – Distributed

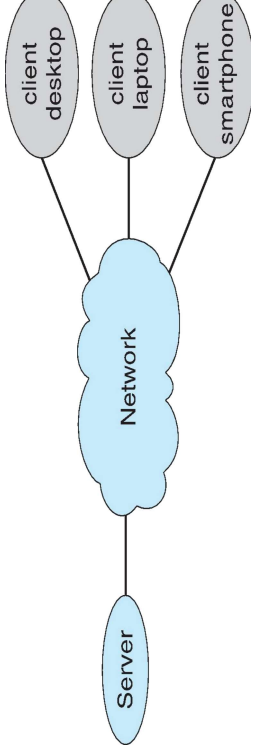
- Distributed computing
  - Collection of separate, possibly heterogeneous, systems networked together
    - **Network** is a communications path, **TCP/IP** most common
      - **Local Area Network (LAN)**
      - **Wide Area Network (WAN)**
      - **Metropolitan Area Network (MAN)**
      - **Personal Area Network (PAN)**
    - **Network Operating System** provides features between systems across network
  - Communication scheme allows systems to exchange messages
  - Illusion of a single system



## Computing Environments – Client-Server

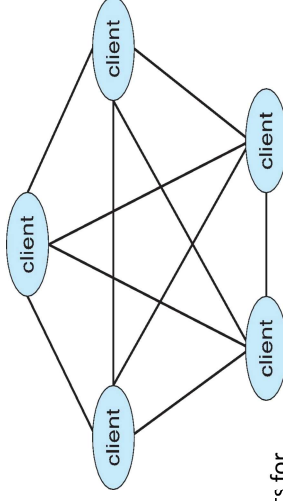
### Client-Server Computing

- Dumb terminals supplanted by smart PCs
- Many systems now **servers**, responding to requests generated by **clients**
- 4 **Compute-server system** provides an interface to client to request services (i.e., database)
- 4 **File-server system** provides interface for clients to store and retrieve files



## Computing Environments - Peer-to-Peer

- Another model of distributed system
- P2P does not distinguish clients and servers
  - Instead all nodes are considered peers
  - May each act as client, server or both
  - Node must join P2P network
    - Registers its service with central lookup service on network, or
    - Broadcast request for service and respond to requests for service via **discovery protocol**
- Examples include Napster and Gnutella, **Voice over IP (VoIP)** such as Skype



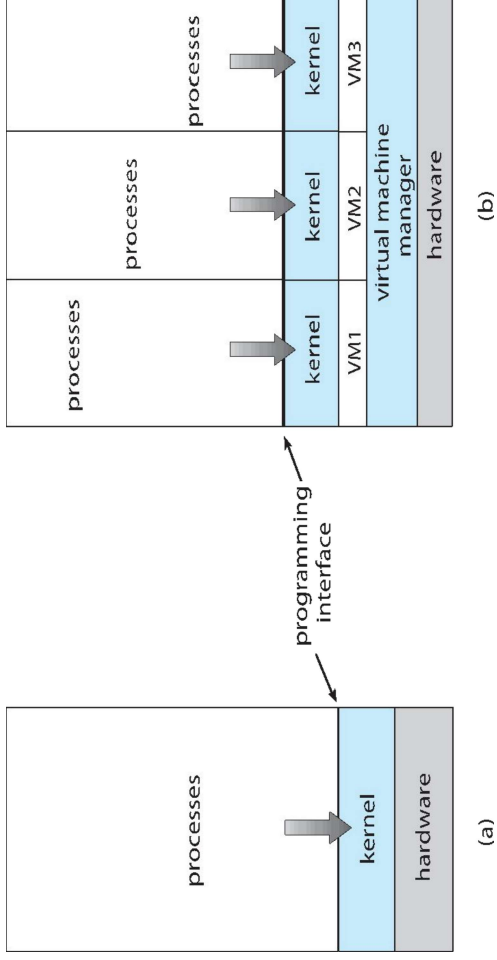
## Computing Environments - Virtualization

- Allows operating systems to run applications within other OSes
  - Vast and growing industry
- **Emulation** used when source CPU type different from target type (i.e. PowerPC to Intel x86)
  - Generally slowest method
  - When computer language not compiled to native code – **Interpretation**
- **Virtualization** – OS natively compiled for CPU, running **guest** OSes also natively compiled
  - Consider VMware running WinXP guests, each running applications, all on native WinXP **host** OS
  - **VMM** (virtual machine Manager) provides virtualization services

## Computing Environments - Virtualization

- Use cases involve laptops and desktops running multiple OSes for exploration or compatibility
  - Apple laptop running Mac OS X host, Windows as a guest
  - Developing apps for multiple OSes without having multiple systems
  - QA testing applications without having multiple systems
  - Executing and managing compute environments within data centers
- VMM can run natively, in which case they are also the host
  - There is no general purpose host then (VMware ESX and Citrix XenServer)

## Computing Environments - Virtualization

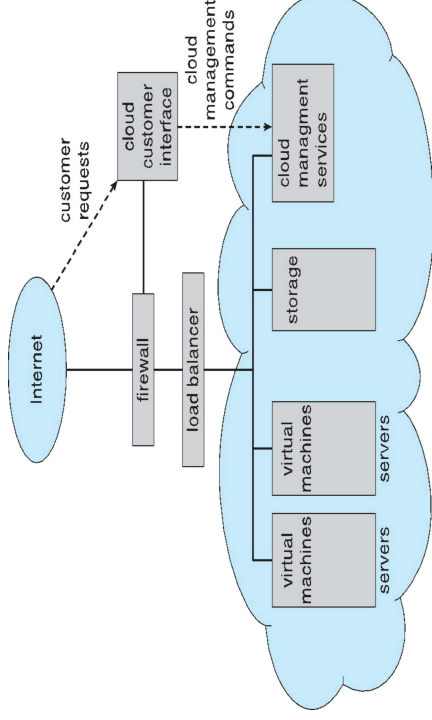


## Computing Environments – Cloud Computing

- Delivers computing, storage, even apps as a service across a network
- Logical extension of virtualization because it uses virtualization as the base for its functionality.
  - Amazon **EC2** has thousands of servers, millions of virtual machines, petabytes of storage available across the Internet, pay based on usage
- Many types
  - **Public cloud** – available via Internet to anyone willing to pay
  - **Private cloud** – run by a company for the company's own use
  - **Hybrid cloud** – includes both public and private cloud components
  - Software as a Service (**SaaS**) – one or more applications available via the Internet (i.e., word processor)
  - Platform as a Service (**PaaS**) – software stack ready for application use via the Internet (i.e., a database server)
  - Infrastructure as a Service (**IaaS**) – servers or storage available over Internet (i.e., storage available for backup use)

## Computing Environments – Cloud Computing

- Cloud computing environments composed of traditional OSes, plus VMMs, plus cloud management tools
  - Internet connectivity requires security like firewalls
  - Load balancers spread traffic across multiple applications

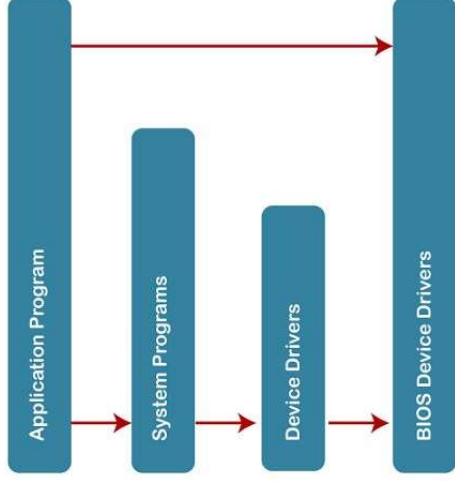


## Computing Environments – Real-Time Embedded Systems

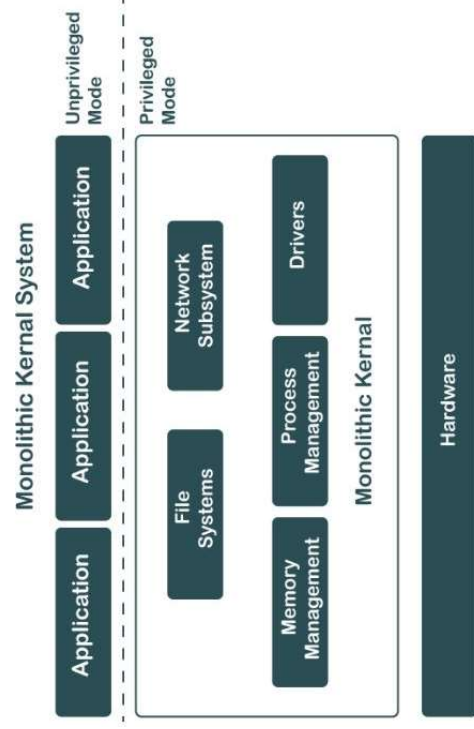
- Real-time embedded systems most prevalent form of computers
  - Vary considerable, special purpose, limited purpose OS, **real-time OS**
  - Use expanding
- Many other special computing environments as well
  - Some have OSes, some perform tasks without an OS
- Real-time OS has well-defined fixed time constraints
  - Processing **must** be done within constraint
  - Correct operation only if constraints met

# Operating System Structure

## Simple Structure

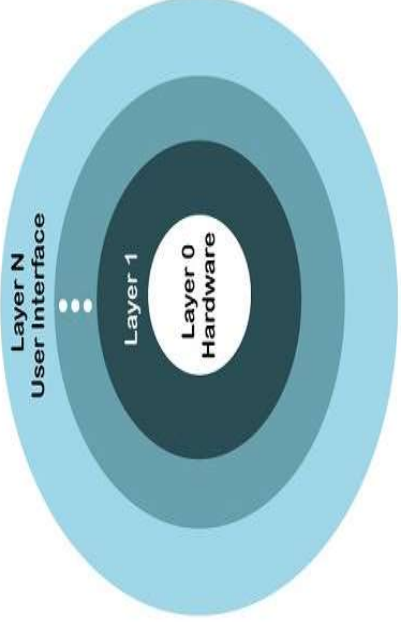


# Operating System Structure



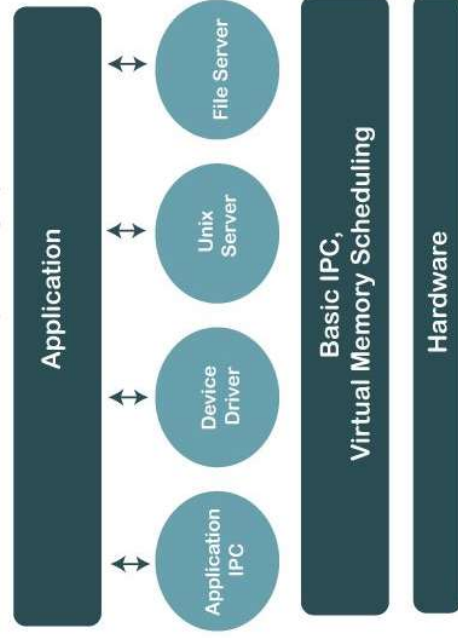
# Operating System Structure

## Layered Structure

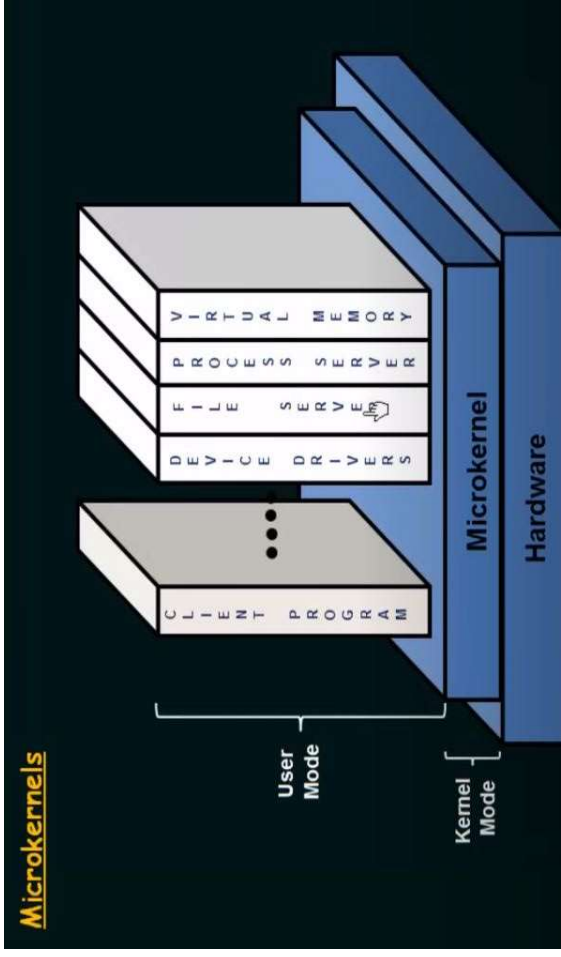


# Operating System Structure

## Microkernel Operating System



# Operating System Structure



## Open-Source Operating Systems

- Operating systems made available in source-code format rather than just binary **closed-source**
- Counter to the **copy protection** and **Digital Rights Management (DRM)** movement
- Started by **Free Software Foundation (FSF)**, which has “copyleft” **GNU Public License (GPL)**
- Examples include **GNU/Linux** and **BSD UNIX** (including core of **Mac OS X**), and many more
- Can use VMM like VMware Player (Free on Windows), Virtualbox (open source and free on many platforms - <http://www.virtualbox.com>)
  - Use to run guest operating systems for exploration

## UNIX History

- The UNIX operating system was born in the late 1960s. It originally began as a one man project led by Ken Thompson of Bell Labs, and has since grown to become the most widely used operating system.
- In the time since UNIX was first developed, it has gone through many different generations and even mutations.
  - Some differ substantially from the original version, like Berkeley Software Distribution (BSD) or Linux.
  - Others, still contain major portions that are based on the original source code.
- An interesting and rather up-to-date timeline of these variations of UNIX can be found at <http://www.levenez.com/unix/history.html>.

## General Characteristics of UNIX as an Operating System (OS)

- **Multi-user & Multi-tasking** - most versions of UNIX are capable of allowing multiple users to log onto the system, and have each run multiple tasks. This is standard for most modern OSs.
- **Over 30 Years Old** - UNIX is over 30 years old and it's popularity and use is still high. Over these years, many variations have spawned off and many have died off, but most modern UNIX systems can be traced back to the original versions. It has endured the test of time. For reference, Windows at best is half as old (Windows 1.0 was released in the mid 80s, but it was not stable or very complete until the 3.x family, which was released in the early 90s).
- **Large Number of Applications** – there are an enormous amount of applications available for UNIX operating systems. They range from commercial applications such as CAD, Maya, WordPerfect, to many free applications.
- **Free Applications and Even a Free Operating System** - of all of the applications available under UNIX, many of them are free. The compilers and interpreters that we use in most of the programming courses here at UMBC (University of Maryland, Baltimore County) can be downloaded free of charge. Most of the development that we do in programming courses is done under the Linux OS.
- **Less Resource Intensive** - in general, most UNIX installations tend to be much less demanding on system resources. In many cases, the old family computer that can barely run Windows is more than sufficient to run the latest version of Linux.
- **Internet Development** - Much of the backbone of the Internet is run by UNIX servers. Many of the more general web servers run UNIX with the Apache web server - another free application.



## Parts of the UNIX OS

- **The Kernel** - handles memory management, input and output requests, and program scheduling. Technically speaking, the kernel *is* the OS. It provides the basic software connection to the hardware. The kernel is very complex and deals with the inner workings of these things, and is beyond the scope of any course.
- **The Shell and Graphical User Interfaces (GUIs)** - basic UNIX shells provides a “command line” interface which allows the user to type in commands. These commands are translated by the shell into something the kernel can comprehend, and then executed by the kernel.
- **The Built-in System Utilities** - are programs that allow a user to perform tasks which involve complex actions. Utilities provide user interface functions that are basic to an operating system, but which are too complex to be built into the shell. Examples of utilities are programs that let us see the contents of a directory, move & copy files, remove files, etc...
- **Application Software & Utilities** – these are not part of the operating system, per se. They are additional programs that are bundled with the OS distribution, or available separately. These can range from additional or different versions of basic utilities, to full scale commercial applications.

## Flavors of UNIX

- These can be grouped into two categories: **Open Source** and **Proprietary**
- **Proprietary:** (redistribution and modification prohibited or restricted; not free)
  - **Solaris** - Access to a Solaris UNIX server (solaris.gl.umbc.edu) via SSH access.
  - **IRIX** - There are a couple of IRIX machines in the basement of the library, as well as server access (irix.gl.umbc.edu) via SSH access.
  - **Mac OS X** - no access currently available at UMBC.
  - and many others...
- **Open Source:** (source code is readily available and free to modify)
  - **FreeBSD** - no access currently available at UMBC.
  - Linux Distributions - access is available in the form of dual-boot PCs scattered throughout the Engineering building. There are also several Linux servers (linux.gl.umbc.edu) through which access is available.
    - **RedHat** (used by UMBC) and the **Fedora Project** (maintained by RedHat)
    - **Mandrake**
    - **Debian**
    - **SuSE**
    - **Slackware**
    - and many others...
- As a side note, Linux is an open source UNIX-based OS that was originally developed in 1991 by Linus Torvalds, a Finnish undergraduate student.

## UNIX Interfaces

- There are really 2 means of connecting to UNIX computers here at UMBC:
  - You can be sitting in front of a dual-bootable PC that you have booted into Linux and logged onto. All of your commands are then being run locally on that computer. When you logon in this manner you have a full GUI environment.
  - You can connect remotely to one of the UNIX servers (whether from home or at the labs). This is often how your projects are suggested to be developed as they are graded on those same servers. When you logon in this manner you have a command line (or text based) environment. You can also open up a command line on local lab machines as well.

## Graphical User Interfaces (GUIs)

- When you logon locally, you are presented with graphical environment.
- You start at a graphical login screen. You must enter your username and password. You also have the option to choose from a couple session types. Mainly you have the choice between Gnome and KDE.
- Once you enter in your username and password, you are then presented with a graphical environment that looks like one of the following...

### Command Line Interface

- You also have access to some UNIX servers as well.
- You can logon from virtually any computer that has internet access whether it be Windows, Mac, or UNIX itself.
- In this case you are communicating through a local terminal to one of these remote servers.
- All of the commands actually execute on the remote server.
- It is also possible to open up graphical applications through this window, but that requires a good bit more setup and software.

### Linux vs. Windows

- OS does not have to use a graphical interface.
- The OS itself (the kernel) is incredibly small.
- The GUI just another application (or set of applications) that can be installed and run on top the existing text-based OS.
- File system differences.
- Windows typically uses FAT32 or NTFS file systems.
- Linux typically uses the ext2 , ext3, ext4` file systems
- Windows lists all drives separately (A:, C:, D:, etc...), with “My Computer” at the highest level.
- UNIX starts its highest level at “/” and drives can be mounted anywhere underneath it.

### What is a Desktop Manager?

- Gnome and KDE are examples of desktop managers. Both of these look a lot like Microsoft Windows.
- They have the equivalent of a Start Menu, have an equivalent of Windows Explorer, and have some sort of control panel.
- The roll of the Desktop Manager is to provide you with the ability to manage all of the details of your system that would otherwise require you to type in a bunch of commands in a terminal window.
- These details include managing your files, launching programs, configuring various aspects of your system, etc.
- It is also worthy to note that the desktop manager is optional. Many older systems did not have a desktop manger that sat in-between the X server and the Window manager.

### What is a Window Manager?

- The Window Manager is a program that manages the placement of Windows on your system. The Window Manager makes it possible to move, resize, and minimize the various programs running on your computer
- KDE handles this functionality as well, whereas Gnome does not directly provide this functionality, but rather relies on an independent window manager to do it for us.
- Think of the Window Manager as the framing around the windows as well as all of the associated functionality that they provide.
- For example, most all window managers can close, minimize, maximize & resize.
- However most UNIX window managers add so much more in the way of functionality. The decoration and customization of these windows under UNIX tends to be much more flexible.
- Many Window Managers also provide other functionality such as shading, sticky/nonsticky, window history, and desktop and workspace manipulations.

### Some Notes on X window, Desktop Managers & Window Managers

- Most UNIX systems can be installed without the GUI.
- The GUI is just another application that runs on top of the operating system.
- There are many implementations of all three of these components.
- It is possible to mix and match implementation and versions of these.
- They need not be alike and need not be all by the same organization.
- This is quite a shift in paradigm from Microsoft and Apple.

### Programming Tools and Utilities Available under Linux

#### • Text Editors

- Xemacs
- Emacs
- Pico
- vi

#### • Compilers

- C compiler - gcc
- C++ compiler - g++
- Java compiler & Java Virtual Machine -  
javac & java

#### • Debuggers

- C / C++ debugger - gdb

#### • Interpreters

- Perl - perl
- Tcl/Tk - tcl & wish

#### • Miscellaneous

- Web Browsers - Mozilla, Netscape, Firefox, and Lynx (lynx is text based)
- Instant Messengers - Gaim
- Email - Netscape is there, but we will learn Pine

### CFS: Completely fair process scheduling in Linux

- The Completely Fair Scheduler (CFS) is a process scheduler which was merged into the 2.6.23 (October 2007) release of the Linux kernel and is the default scheduler. It handles CPU resource allocation for executing processes, and aims to maximize overall CPU utilization while also maximizing interactive performance.
- In contrast to the previous O(1) scheduler used in older Linux 2.6 kernels, the CFS scheduler implementation is not based on run queues. Instead, a red-black tree implements a "timeline" of future task execution. Additionally, the scheduler uses nanosecond granularity accounting, the atomic units by which an individual process' share of the CPU was allocated (thus making redundant the previous notion of timeslices). This precise knowledge also means that no specific heuristics are required to determine the interactivity of a process, for example.
- Like the old O(1) scheduler, CFS uses a concept called "sleepers fairness", which considers sleeping or waiting tasks equivalent to those on the runqueue. This means that interactive tasks which spend most of their time waiting for user input or other events get a comparable share of CPU time when they need it.

### Comparison of cpu scheduling of Linux and Windows OS

| Windows                                                                                       | Linux                                                                                                    |
|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| 1) Process                                                                                    | 1) Process is called a Task                                                                              |
| a) Address space, handle table, statistics and at least one thread                            | a) Basic Address space, handle table, statistics                                                         |
| b) No inherent parent/child relationship                                                      | b) Parent/child relationship                                                                             |
| 2) Threads                                                                                    | c) Basic scheduling unit                                                                                 |
| a) Basic scheduling unit                                                                      | 2) Threads                                                                                               |
| b) Fibers – cooperative user-mode threads                                                     | a) No threads per-se                                                                                     |
|                                                                                               | b) Tasks can act like Windows threads by sharing handle table, PID and address space                     |
|                                                                                               | c) P-Threads – cooperative user-mode threads                                                             |
| 3) Two scheduling classes                                                                     | 3) Has 3 scheduling classes                                                                              |
| a) "Real time" (fixed) – priority 16-31                                                       | a) Normal – priority 100-139                                                                             |
| b) Dynamic – priority 1-15                                                                    | b) Fixed Round Robin – priority 0-99                                                                     |
|                                                                                               | c) Fixed FIFO – priority 0-99                                                                            |
| 4) Most threads run in variable priority levels                                               | 4) Most threads use a dynamic priority policy                                                            |
| a) Priorities 1-15;                                                                           | a) Normal class – similar to the classic UNIX scheduler                                                  |
| b) A newly created thread starts with a base priority                                         | b) A newly created thread starts with a base priority                                                    |
| c) Threads that complete I/O operations experience priority boosts (but never higher than 15) | c) Threads that block frequently (I/O bound) will have their priority gradually increased                |
| d) A thread's priority will never be below base priority                                      | d) Threads that always exhaust their time slice (CPU bound) will have their priority gradually decreased |

## Comparison of CPU scheduling of Linux and Windows OS

| Windows                                                                                                                                                                                                                                                                                                                                                   | Linux                                                                                                                                                                                                                                                                                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>5) Real time scheduling in windows.<br/>Windows xp supports static round-robin scheduling policy for threads with priorities in real-time range (16-31)</p> <ul style="list-style-type: none"> <li>a) Threads run for up to one quantum.</li> <li>b) Quantum is reset to full turn on preemption.</li> <li>c) Priorities never get boosted.</li> </ul> | <p>5) Real time scheduling in Linux.<br/>Linux supports two static priority scheduling policies: Round-robin and FIFO (first in, first out)</p> <ul style="list-style-type: none"> <li>a) Selected with the scheduler( ) system call</li> <li>b) Use static priority values in the range of 1 to 99</li> <li>c) Executed strictly in order of decreasing static priority</li> </ul> |
| <p>6) Some System calls and DPC/APC handling can cause priority inversion</p>                                                                                                                                                                                                                                                                             | <p>6) Long running system calls can cause priority-inversion</p>                                                                                                                                                                                                                                                                                                                    |
| <p>7) Scheduling timeslices in windows<br/>The thread time slice (quantum) is 10ms-120ms</p> <ul style="list-style-type: none"> <li>a)When quanta can vary, has one of 2 values</li> </ul>                                                                                                                                                                | <p>7) Scheduling timeslices in Linux.<br/>The thread quantum is 10ms-200ms</p> <ul style="list-style-type: none"> <li>a)Default is 100ms</li> <li>b)Varies across entire range based on priority, which is based on interactivity level</li> </ul>                                                                                                                                  |
| <p>8) Scheduler load balancing is not possible.</p>                                                                                                                                                                                                                                                                                                       | <p>8) Scheduler load balancing is possible.</p>                                                                                                                                                                                                                                                                                                                                     |